rsedit — Rapporto sui problemi del codice

Le schede dell'analisi della revisione `08a853d`, verificate sulla revisione `3d846f1`

Indice

1 Che cosa contiene questo rapporto 2
  1.1 Le schede dell'interprete 2
  1.2 Come leggere una scheda 2
  1.3 Come è stato condotto l'esame 2
  1.4 Riepilogo 3
2 Buffer, modifiche e annullamento 5
3 Tastiera, comandi e primitive dell'editor 6
4 Minibuffer 8
5 File e salvataggio 9
6 Avvio, configurazione e registro 10
7 Lavoro in background 11
8 Modi, sintassi e colorazione 12
9 Ricerca, sostituzione e liste di risultati 13
10 Comandi di shell 15
11 Disegno e terminale 16
12 La finestra grafica 18
13 Documentazione, commenti, codice morto, build e test 19
14 Appendici 21
  14.1 Le schede risolte 21
  14.2 Differenze da Emacs che non sono difetti 22
  14.3 Parti esaminate e trovate solide 22

1 Che cosa contiene questo rapporto

Questo documento elenca i problemi trovati analizzando il codice di rsedit. È separato di proposito dal manuale di architettura, `architettura-editor.typ`. Il manuale descrive come il codice funziona e resta valido anche dopo una correzione; questo rapporto descrive dove il codice non fa ciò che dovrebbe. Quando un problema viene risolto, la sua scheda passa nell'appendice delle schede risolte, con il commit che lo ha risolto.

Ogni problema ha un identificativo stabile, formato dalla sigla dell'area e da un numero (`BUF-2`, `CMD-1`…), che si può citare in un commit o in una discussione; nel codice lo si cita solo in un `TODO(ID)`, e solo finché la scheda è aperta. Le schede sono generate da un'unica tabella di dati all'inizio del sorgente: per aggiungerne una, o per segnarne una come risolta, basta modificare quella tabella, e il riepilogo si aggiorna da solo.

1.1 Le schede dell'interprete

Le schede delle aree LET (il lettore) e INT (il valutatore e la libreria di base) riguardano l'interprete, che da `6d30567` è il crate esterno `risp`. Sono passate nel rapporto del suo repository, `doc/rapporto-problemi.typ`, con gli stessi identificativi. Le schede di qui che vi rimandano lo dicono nel testo, per esempio «INT-2, nel rapporto di risp». rsedit usa la revisione di risp fissata nel suo `Cargo.lock`, oggi `b2d000b`: una scheda che risp risolve dopo quella revisione resta un difetto di rsedit finché la dipendenza non si aggiorna.

1.2 Come leggere una scheda

Ogni scheda riporta:

- **Gravità**, secondo la scala qui sotto.
- **Dove**: i file e le funzioni, o i tipi, in cui sta il difetto. I percorsi senza prefisso sono relativi a `core/src/`; quelli che cominciano con `src/` riguardano la TUI e la GUI, quelli con `core/lisp/` i moduli Lisp.
- **Verifica**: «verificato» se il comportamento è stato riprodotto eseguendo il codice (il testo dice come, e su quale revisione), «dal codice» se la conclusione viene dalla sola lettura.
- **Cosa succede**, il meccanismo del difetto; **Come riprodurlo**, quando serve; **Effetto**, ciò che l'utente vede; **Correzione proposta**.

Tutte le schede dell'elenco sono aperte; le risolte sono nell'appendice.

Gravità	Significato
ALTA	L'editor termina, si blocca senza via d'uscita o perde dati; oppure un contenuto esterno può far eseguire azioni al terminale; oppure l'editor finisce in uno stato da cui si esce solo riavviando.
MEDIA	Un comportamento sbagliato o un rallentamento di secondi in un uso realistico, senza perdita di dati.
BASSA	Casi limite, incoerenze, codice morto, commenti e documentazione superati.

1.3 Come è stato condotto l'esame

Il codice della revisione `08a853d` è stato letto per intero: l'interprete (allora `core/src/lisp`, ora il crate `risp`), la libreria dell'editor (`core/src`), la TUI (`src/`) e i moduli Lisp (`core/lisp`). Ogni comportamento sospetto è stato riprodotto, dove possibile, con piccoli programmi esterni che usano `rsedit_core` come una libreria:

- un **esecutore di scenari** che valuta espressioni Lisp, preme tasti attraverso il percorso reale dei tasti, attende i lavori di background e stampa buffer, area messaggi ed evidenziazioni della fotografia dello schermo;

- un **fuzzer delle primitive**, che chiama ciascuna delle 628 funzioni registrate (tranne quelle che toccano file o processi esterni) con 807 combinazioni di argomenti anomali (zero, negativi, 10^{23}, infinito, NaN, stringhe vuote e non ASCII, liste puntate, vettori, mappe, lambda), una volta su un buffer vuoto e una su un buffer con testo multiriga, caratteri non ASCII, tab, una regione attiva e finestre divise.

I fuzzer del lettore e delle forme speciali, che hanno trovato le schede dell'interprete, sono descritti nel rapporto di risp. Le misure di tempo sono state prese con build di release su una macchina condivisa, quindi vanno lette come ordini di grandezza.

1.3.1 La verifica sulla revisione `3d846f1`

Ogni scheda è stata poi verificata sulla revisione `3d846f1`. Il codice che descrive è stato riletto, e la storia di git ha detto quale commit lo ha cambiato. Dove la scheda dice «verificato», la riproduzione è stata eseguita di nuovo, con lo stesso esecutore di scenari o con due editor veri in un terminale virtuale. Le misure di tempo e di memoria non sono state ripetute dove il codice misurato non è cambiato, e la scheda lo dice. Il campo **Dove** nomina ora file e funzioni, non righe.

Delle 51 schede di rsedit, 11 sono risolte. Sette lo sono in parte e restano aperte con il testo aggiornato a ciò che manca: BUF-1, CMD-1, CMD-3, CMD-6, MIN-3, FIL-5 e DOC-1. La verifica ne ha trovate quattro nuove: AVV-6, BKG-2, RIC-7 e DOC-5. Le schede trovate dopo, nei passi della documentazione che riscrivono il codice, dicono nella verifica dove sono state riprodotte, o che vengono dalla lettura del codice: RIC-8, BUF-4, CMD-7, CMD-8, BKG-3, SHL-5, MIN-4, RIC-9, MIN-5, FIL-6, SIN-7, SIN-8, SIN-9, AVV-7, AVV-8, GUI-1 e CMD-9. RIC-10 è stata trovata, con un test, mentre si correggeva la visita dei risultati per la directory di lavoro, e risolta nello stesso commit: è tra le risolte.

1.4 Riepilogo

> Sono aperti **34 problemi**: **0 di gravità alta**, 3 di gravità media e 31 di gravità bassa; 17 sono stati riprodotti eseguendo il codice. Altri 39 sono stati risolti dopo l'analisi, e sono nell'appendice.

Non restano problemi di gravità alta: dei quattro trovati, due riguardavano il terminale, che un file poteva comandare (TUI-1) e che un panic o un errore lasciavano in raw mode (TUI-2), e due facevano perdere lavoro senza dirlo, un salvataggio fallito (FIL-1) e due prompt aperti insieme (MIN-1). Le tre correzioni trasversali che l'analisi proponeva ci sono, nell'editor: nel WorkManager un panic ritira solo il lavoro che l'ha causato (BKG-1), una guardia ripristina il terminale (TUI-2), e le larghezze guidate dall'utente hanno un limite, mentre i conteggi si fermano al bordo del buffer (CMD-3). Il limite manca ancora nell'interprete (INT-2 e INT-13, nel rapporto di risp).

La tabella seguente elenca i problemi aperti, dal più grave; l'identificativo rimanda alla scheda.

ID	Problema	Gravità	Verifica
TUI-3	Tab non espansi: cursore, colori e riempimento sfasati	MEDIA	verificato
TUI-4	CJK ed emoji contati come una colonna	MEDIA	dal codice
TUI-5	`Color::parse`: panic su colori non ASCII	MEDIA	verificato
BUF-3	`GapBuffer::clone` è `todo!()`; `find_forward` underflow	BASSA	dal codice
BUF-4	Sola lettura: kill, verbi, commenti e rettangoli tacciono; `kill-rectangle` conserva il blocco	BASSA	verificato
CMD-4	`parse_key`: un solo prefisso; `define-key` fallisce in silenzio	BASSA	verificato
CMD-5	`C-u 3 x` inserisce una sola `x`	BASSA	verificato
CMD-6	`kill-word`, `delete-char` e simili: un conteggio negativo vale 0	BASSA	dal codice
CMD-7	Windows: `insert-pasted-text` non converte CR e CRLF	BASSA	dal codice
CMD-8	`close-floating-window` senza flottanti: focus su una finestra che può non esserci	BASSA	verificato

ID	Problema	Gravità	Verifica
CMD-9	vim: entrare e uscire da Insert costa quanto il file	BASSA	verificato
MIN-4	MODE nil a `minibuffer-read`: il prompt senza i suoi tasti	BASSA	verificato
FIL-4	Corsa tra copia del testo e azzeramento di «modificato»	BASSA	dal codice
FIL-5	Autosave: versione segnata anche se la scrittura fallisce	BASSA	dal codice
FIL-6	Copie in `auto-save-directory`: `/a!/b` e `/a/!b` hanno lo stesso nome	BASSA	verificato
AVV-6	Due editor insieme: il secondo tronca il registro del primo, e le righe si sovrappongono	BASSA	verificato
AVV-8	Senza cache, niente `rsedit.log`, in silenzio	BASSA	dal codice
BKG-2	Un panic ritira il lavoro per sempre: colorazione o salvataggio automatico fermi senza avviso	BASSA	dal codice
BKG-3	`setenv`: le letture dell'ambiente fuori da `std` non sono escluse	BASSA	dal codice
SIN-4	`make-mode` ricrea il modo, perdendo legami e hook	BASSA	dal codice
SIN-5	Facce: registro globale senza limite, indice `u16`	BASSA	dal codice
RIC-5	Risultati per numero di riga, non per marcatore	BASSA	verificato
RIC-7	Liste di occur e di grep in due strisce separate	BASSA	verificato
RIC-8	`query-replace` lasciato: sessione ed evidenziazione restano	BASSA	verificato
RIC-9	BUFFER nil a `results-put` e `results-select`: il buffer `nil`	BASSA	verificato
SHL-4	`manpage`: nome interpolato senza quoting nella shell	BASSA	verificato
SHL-5	All'uscita il `KILL` di un comando fermato non parte	BASSA	verificato
TUI-6	Finestre flottanti senza sintassi e overlay	BASSA	dal codice
TUI-7	`%p` = `All` solo con una riga	BASSA	dal codice
TUI-8	Niente schermo alternativo: all'uscita lo schermo viene cancellato	BASSA	dal codice
GUI-1	GUI: un aggiornamento ogni 250 ms anche da ferma	BASSA	dal codice
DOC-1	Commenti e docstring superati	BASSA	dal codice
DOC-2	`Debug` di `EditorState` è `todo!()`, `PartialEq` è `unreachable!()`	BASSA	dal codice
DOC-5	`free_buffer_name` mai usato; due `if` da unire; 41 blocchi da formattare	BASSA	verificato

Area	Descrizione	Alta	Media	Bassa	Risolte
BUF	Buffer, modifiche e annullamento	0	0	2	2
CMD	Tastiera, comandi e primitive dell'editor	0	0	6	3
MIN	Minibuffer	0	0	1	4
FIL	File e salvataggio	0	0	3	3
AVV	Avvio, configurazione e registro	0	0	2	6
BKG	Lavoro in background	0	0	2	1
SIN	Modi, sintassi e colorazione	0	0	2	7
RIC	Ricerca, sostituzione e liste di risultati	0	0	4	6
SHL	Comandi di shell	0	0	2	3
TUI	Disegno e terminale	0	3	3	2
GUI	La finestra grafica	0	0	1	0
DOC	Documentazione, commenti, codice morto, build e test	0	0	3	2

Tabella 1: I problemi aperti per area e gravità, e quelli risolti

2 Buffer, modifiche e annullamento

BUF-3 — `GapBuffer::clone` è `todo!()` e `find_forward` va in underflow su un buffer vuoto

Gravità	BASSA
Dove	`buffer/gap_buffer.rs` (`impl Clone for GapBuffer`, `find_forward`), `buffer/buffer_trait.rs` (`BufferTrait`, che richiede `Clone`)
Verifica	Dalla lettura del codice; oggi nessun percorso li raggiunge (`find_forward` è usata solo nei test).

Cosa succede. `BufferTrait` richiede `Clone`, ma l'implementazione per `GapBuffer` è un `todo!()`: il primo che clonerà un buffer manderà l'editor in panic. `find_forward` calcola `self.len() - 1`, che su un testo vuoto va in underflow (panic in debug) e poi arriva a un `unreachable!()`.
Correzione proposta. Derivare `Clone`, oppure togliere il vincolo dal trait; in `find_forward` usare `checked_sub`.

BUF-4 — In un buffer in sola lettura molti comandi non dicono nulla, e `kill-rectangle` conserva un blocco che non ha tolto

Gravità	BASSA
Dove	`primitives/killing.rs` (`cut_out`, e le kill che la usano), `primitives/verbs.rs` (`replace_span` e i verbi), `primitives/comments.rs` (`apply`), `primitives/rectangle.rs` (`cut`, `insert_at_edge`, `insert_block` e i comandi che li usano), `primitives/macros.rs` (`kmacro-insert-counter`)
Verifica	Riprodotto con un test temporaneo durante il passo 16 della documentazione, sul codice di `9fcd84b` con i passi 12–15, che non cambiano il comportamento. In un buffer in sola lettura `kill-line`, `kill-word`, `upcase-word`, `transpose-chars`, `join-line`, `comment-dwim` e `zap-to-char` non cambiano il testo e lasciano vuota l'area messaggi. `kill-rectangle` restituisce 3 e lascia il testo com'è, e dopo `(killed-rectangle)` restituisce le tre righe del blocco; `delete-rectangle`, `open-rectangle` e `string-rectangle` restituiscono 3 senza aver cambiato nulla; `kmacro-insert-counter` segnala l'errore «This buffer is read-only». Che il contatore sia già avanzato viene dalla lettura del codice.

Cosa succede. Le due porte rifiutano la modifica e restituiscono `false`, e dirlo all'utente tocca a chi le chiama, con `edited`, che scrive «Buffer is read-only». Lo fanno le primitive di `edits.rs`, `kill-region`, `yank`, `yank-pop` e `indent-line-to`. Le kill che passano da `cut_out` ne ricevono una stringa vuota e la danno all'anello, che la ignora; i verbi e i commenti rispondono nil; i comandi dei rettangoli scartano il risultato delle porte e rispondono con il numero di righe. `kill-rectangle` legge il blocco prima di cancellarlo, e lo conserva per `yank-rectangle` anche se la cancellazione è stata rifiutata. `kmacro-insert-counter` fa avanzare il contatore prima di inserire, e segnala un errore al posto del messaggio.
Effetto. (1) Un tasto che non fa nulla e non dice nulla: in una vista in sola lettura, come dired o l'aiuto, sembra un editor rotto e non un buffer protetto. (2) Dopo `kill-rectangle` in un buffer in sola lettura, `yank-rectangle` in un altro buffer inserisce un blocco che non è mai stato tolto, cioè lo duplica.
Correzione proposta. Far usare `edited` a tutti i comandi che modificano, a partire da `cut_out` e da `replace_span`; nei rettangoli fermarsi al primo rifiuto, senza conservare il blocco né spostare il punto; in `kmacro-insert-counter` inserire prima di far avanzare il contatore.

3 Tastiera, comandi e primitive dell'editor

CMD-4 — Le combinazioni di modificatori in `define-key` sono limitate e un errore va solo nel registro

> **Gravità** BASSA
> **Dove** `input.rs` (`parse_key`), `primitives/keymaps.rs` (`define-key`)
> **Verifica** Riprodotto sulla revisione `3d846f1`: `"M-C-x"` e `"C-S-a"` danno `nil`, `"C-M-x"` dà `t`.

Cosa succede. `parse_key` riconosce un solo prefisso tra `C-M-`, `C-`, `M-` e `S-`, quindi `M-C-x` (stesso tasto di `C-M-x`) e le combinazioni con Shift non si possono scrivere. Una sequenza non valida o un modo inesistente fanno restituire `nil` a `define-key`, con il motivo scritto solo nel registro. La docstring di `define-key` e la pagina `keymaps` ora dicono quali combinazioni non sono tasti.
Correzione proposta. Analizzare i prefissi in un ciclo, in qualsiasi ordine, e segnalare l'errore con un errore Lisp.

CMD-5 — `self-insert` ignora l'argomento prefisso

> **Gravità** BASSA
> **Dove** `input.rs` (`fill_default_keymaps`: il legame è `(self-insert "x")`, una forma che fornisce già i suoi argomenti), `editor/keys.rs` (`EditorState::handle_key_event`), `primitives/edits.rs` (`self-insert`)
> **Verifica** Riprodotto sulla revisione `3d846f1`: `C-u 3 x` inserisce una `x`.

Cosa succede. Una forma che fornisce già i suoi argomenti non riceve l'argomento prefisso, e `self-insert` non lo legge. La pagina `editing` ora lo dice tra i conteggi; in Emacs le `x` sono tre.
Correzione proposta. Legare i caratteri a un comando che legge il tasto premuto (come `self-insert-command` di Emacs), oppure passare il prefisso alle forme di `self-insert`.

CMD-6 — Un conteggio negativo non inverte i comandi che cancellano

> **Gravità** BASSA
> **Dove** `primitives/args.rs` (`repeat_count`), e i comandi che lo usano: `delete-char` in `primitives/edits.rs`, `kill-word` e `backward-kill-word` in `primitives/killing.rs`, `kill-sexp` e `backward-kill-sexp` in `primitives/sexp.rs`
> **Verifica** Riprodotto sulla revisione `3d846f1`: in `foo bar`, `(kill-word -1)` dalla fine e `(delete-char -1)` da metà non cambiano il testo.

Cosa succede. `repeat_count` trasforma un conteggio negativo in 0, quindi i comandi che lo usano non fanno nulla, mentre in Emacs agiscono all'indietro. Due parti della scheda sono risolte: da `4506432` una risposta non numerica a una domanda numerica viene chiesta di nuovo, e da `cf0d8ac` gli spostamenti (caratteri, parole, righe, paragrafi, espressioni) usano `movement_count` e con un conteggio negativo vanno all'indietro.
Correzione proposta. Usare `movement_count` anche nei comandi che cancellano, cancellando all'indietro con un conteggio negativo.

CMD-7 — Su Windows un incolla tiene i ritorni a capo CR, e lì quattro test dell'incolla fallirebbero

> **Gravità** BASSA
> **Dove** `primitives/edits.rs` (la variante per Windows di `normalize_line_endings`, che usa `insert-pasted-text`), `tests/paste_tests.rs`
> **Verifica** Dalla lettura del codice: i test non girano su Windows, e la verifica dei passi compila soltanto per quel sistema.

Cosa succede. `insert-pasted-text` converte in LF ogni CRLF e ogni CR isolato prima di inserire, perché l'indice delle righe conta solo `\n`. Su Windows invece `normalize_line_endings` restituisce il testo com'è, e nessun commento dice perché. La GUI passa a `handle_paste` gli appunti del sistema, che su Windows hanno di solito i fine riga CRLF. `paste_tests` non esclude Windows, e lì `a_paste_with_crlf_endings_becomes_lines`,

`a_paste_with_bare_carriage_returns_becomes_lines`, `a_mix_of_endings_all_become_lines` e `a_trailing_carriage_return_opens_a_line` fallirebbero.

Effetto. Su Windows un testo di più righe incollato lascia un CR alla fine di ogni riga, e un testo con CR isolati diventa una riga sola.

Correzione proposta. Decidere che cosa vuole Windows: convertire come sugli altri sistemi, togliendo la variante, oppure scriverne il motivo e limitare i quattro test agli altri sistemi.

CMD-8 — `close-floating-window` senza finestre flottanti sposta il focus sulla finestra 0, anche quando non esiste

Gravità	BASSA
Dove	`primitives/windows.rs` (`close-floating-window`), `managers/windows.rs` (`Windows::set_focused`, che non controlla l'id)
Verifica	Riprodotto con un piccolo programma temporaneo durante il passo 17 della documentazione, sul codice di `9fcd84b` con i passi 12–16, che non cambiano il comportamento. Dopo `(split-window-right)` e `(delete-window)` resta la finestra 1, che ha il focus; `(close-floating-window)` restituisce nil, e da lì `(selected-window)` risponde 0, `(window-buffer)` nil e `(split-window-below)` nil, con `(count-windows)` sempre 1; `(other-window)` non sposta il focus. `(select-window 1)` lo riporta sulla finestra che c'è, e la divisione torna a funzionare.

Cosa succede. La primitiva toglie la flottante più recente e rende il focus alla finestra che questa ricorda. Quando non ce n'è nessuna, l'id da rendere manca e la primitiva mette il focus sull'id 0, che `Windows::set_focused` accetta senza guardare se esiste. La docstring dice che senza flottanti non fa nulla. In `debug.lisp`, `backtrace-dismiss` la chiama con il tasto `q` del modo del buffer `*Backtrace*`.

Effetto. Il focus resta su una finestra che non c'è: non si può dividere, e `other-window` non ne esce.

Correzione proposta. Non toccare il focus quando non c'è una flottante da chiudere, e controllare l'id in `set_focused`.

CMD-9 — In vim ogni sessione di Insert copia e confronta il buffer intero

Gravità	BASSA
Dove	`core/lisp/features/vim.lisp` (`vim--begin-insert`, che ricorda `(buffer-string)`, e `vim--finish-insert`, che lo confronta con `string-diff`)
Verifica	Misurato con un test temporaneo durante il passo 21 della documentazione, compilato in release: su un buffer di 100 KB `i`, `x` ed Esc costano 1,3 ms, su uno di 2 MB 47 ms, mentre un movimento costa 0,2 ms in tutti e due.

Cosa succede. Per sapere che cosa la sessione ha cambiato, e farlo ripetere a `.`, `vim--begin-insert` copia il testo del buffer intero in una proprietà del buffer, e `vim--finish-insert` ne fa un'altra copia e confronta le due con `string-diff`. Il costo è proporzionale al file, non a ciò che si è scritto, e lo pagano ogni `i`, `a`, `o`, `c` e Replace, anche quando la sessione non cambia nulla; la copia resta in memoria per tutta la sessione.

Effetto. In un file di qualche megabyte entrare e uscire da Insert ha un ritardo visibile, che cresce con il file: decine di millisecondi a 2 MB, circa duecento a 8 MB. Sui file piccoli non si nota.

Correzione proposta. Ricordare l'intervallo toccato, dal punto d'ingresso e dalle modifiche che le porte registrano (l'undo le conosce già), invece del testo intero; oppure confrontare solo le righe tra la prima e l'ultima modificata.

4 Minibuffer

MIN-4 — Un `minibuffer-read` con MODE nil apre il prompt nel modo `nil`, dove Invio ed Esc non rispondono

Gravità	BASSA
Dove	`primitives/minibuffer.rs` (`default-minibuffer-prompt`, che prende ogni simbolo come il nome del modo)
Verifica	Riprodotto con un test temporaneo durante il passo 18 della documentazione, sul codice di `71e1554`, che non cambia il comportamento. Dopo `(minibuffer-read "P:" (lambda (s) (setq got s)) nil nil nil "chiave")`, `(major-mode)` nel prompt risponde `nil`; `h`, `i` e Invio lasciano nel prompt `hi` e un a capo senza chiamare ON-CONFIRM, ed Esc non lo chiude. Con MODE omesso, o `'minibuffer-mode`, la stessa sequenza conferma.

Cosa succede. Il prompt predefinito legge MODE come il nome di un modo quando è un simbolo o una stringa, e nil è il simbolo `nil`: il buffer del prompt si apre nel modo `nil`, che non esiste, senza i tasti di `minibuffer-mode`. La docstring dice che MODE è `minibuffer-mode` se manca, e chi vuole dare KEY senza MODE passa nil, come si fa in Lisp con un argomento facoltativo. Nessun modulo fornito con l'editor lo fa.

Effetto. Il prompt non si conferma né si chiude con i suoi tasti, Invio vi scrive un a capo, e nemmeno `C-g` lo chiude, perché anche lui è un tasto di `minibuffer-mode`: ne esce solo chi sa chiudere il buffer `*Minibuffer*` in un altro modo.

Correzione proposta. Prendere nil come assente, con `args::optional_name`, come fanno le altre primitive per un nome facoltativo, e un test con MODE nil e KEY.

5 File e salvataggio

FIL-4 — Una modifica fatta durante il salvataggio può restare non salvata ma non segnata

Gravità	BASSA
Dove	`primitives/files.rs` (`save-buffer`, `save-buffer--overwrite`, `write_buffer`; `write-file` segue lo stesso schema)
Verifica	Dalla lettura del codice.

Cosa succede. Il testo viene copiato sotto il lock, poi scritto senza lock, poi `is_modified` viene azzerato con un secondo lock, senza guardare `version`. Una modifica arrivata nel frattempo (da un lavoro Lisp in background o da un callback) resta nel buffer ma il buffer risulta salvato. Il salvataggio automatico non ha il problema: ricorda la versione che ha copiato.
Correzione proposta. Ricordare la `version` copiata e azzerare `is_modified` solo se è ancora quella.

FIL-5 — Il salvataggio automatico segna come salvato anche un tentativo fallito

Gravità	BASSA
Dove	`jobs/autosave.rs` (`EditorState::write_auto_save`: `auto_saved_at` impostato anche dopo un errore)
Verifica	Dalla lettura del codice: il commento lo dice, «Recorded whether or not the write worked».

Cosa succede. Se la scrittura della copia fallisce per un motivo passeggero, la versione viene comunque registrata, e non si riprova finché il buffer non cambia. La prima metà della scheda è risolta da `1aa5cf7`: `auto_save_for` guarda file, sola lettura, modifica e versione già salvata prima di copiare il testo, e ogni giro guarda tutti i buffer e ne scrive al più quattro.
Correzione proposta. Registrare la versione solo dopo una scrittura riuscita.

FIL-6 — Con `auto-save-directory`, due percorsi diversi possono avere la stessa copia

Gravità	BASSA
Dove	`jobs/autosave.rs` (`flatten`, usata da `auto_save_path`)
Verifica	Riprodotto con un test temporaneo durante il passo 19 della documentazione, sul codice di `20f7221`, che non cambia il comportamento. `auto_save_path("/a!/b", Some("/d"))` e `auto_save_path("/a/!b", Some("/d"))` rispondono entrambe `/d/#!a!!!b#`.

Cosa succede. Per fare del percorso un solo nome di file, `flatten` cambia ogni separatore (`/`, `\`, `:`) in `!` e raddoppia ogni `!`. Così `/a!b/c` e `/a/b/c` restano distinti, ma un `!` accanto a un separatore no: un `!` seguito da un separatore e un separatore seguito da un `!` danno entrambi `!!!`. Il commento della funzione diceva che due percorsi diversi non possono coincidere. È lo schema di Emacs, che ha lo stesso limite. Senza `auto-save-directory` la copia sta accanto al file, con il solo nome, e il problema non c'è.
Effetto. Due buffer di file come `/a!/b` e `/a/!b` scrivono la stessa copia, e il ripristino di uno può dare il lavoro dell'altro.
Correzione proposta. Una codifica che due percorsi non possano condividere, come quella degli URL per il `%` e i separatori, o un hash del percorso nel nome; un test con i due percorsi di sopra.

6 Avvio, configurazione e registro

AVV-6 — Due editor aperti insieme scrivono lo stesso `rsedit.log` e si cancellano le righe a vicenda

Gravità	BASSA
Dove	`editor/log.rs` (`EditorState::enable_log_file`, con `File::create`), `managers/log.rs` (`Log::flush`, che scrive dalla posizione a cui era arrivato), `src/main.rs` (`main`, con un nome fisso nella directory della cache)
Verifica	Riprodotto sulla revisione `3d846f1` con due editor veri e la stessa directory di cache: all'avvio il secondo tronca il file e lo riscrive da capo; poi una riga del secondo («no translation for the key F(5)») e una del primo («Keymap not bound ») finiscono nello stesso punto, e il file dice «Keymap not bound » seguito dal resto dell'altra, «key F(5)».

Cosa succede. Il registro è sempre `rsedit.log` nella directory della cache, aperto con `File::create`, che tronca il file, e senza `O_APPEND`. Un secondo editor tronca quindi il file del primo e ci scrive dall'inizio; il primo continua a scrivere dalla posizione a cui era arrivato, sopra le righe dell'altro. La pagina `diagnostics` dice che il file ricomincia a ogni avvio, non che due editor lo condividono.

Effetto. Con due editor aperti il registro su disco mescola le righe dei due e ne perde una parte: proprio il file che serve quando un editor si è chiuso da solo.

Correzione proposta. Un file per processo (con il PID nel nome, e un collegamento all'ultimo), oppure l'apertura in append con un blocco sul file, che dica al secondo editor che il registro è già in uso.

AVV-8 — Senza la directory della cache l'editor gira senza file di registro, e non lo dice

Gravità	BASSA
Dove	`src/main.rs` (`main`, che ignora l'errore di `app_dir` e quello di `enable_log_file`)
Verifica	Dalla lettura del codice, durante il passo 20 della documentazione.

Cosa succede. `main` attiva il file di registro solo se `app_dir` trova, o crea, la directory della cache, e scarta con `let _` l'errore di `enable_log_file`. In tutti e due i casi l'editor parte con il solo registro in memoria (`*Log*`), senza un messaggio nell'area dei messaggi né una riga nel registro stesso.

Effetto. Su una macchina con la cache non scrivibile, o con `RSEDIT_CACHE_DIR` sbagliata, il file che la pagina `diagnostics` indica non esiste, e chi lo cerca dopo un crash non trova nulla e non sa perché.

Correzione proposta. Scrivere nel registro in memoria, e nell'area dei messaggi, che il file non si è potuto aprire, con il percorso e l'errore.

7 Lavoro in background

BKG-2 — Un lavoro dell'editor che va in panic sparisce per tutta la sessione, e lo dice solo il registro

Gravità	BASSA
Dove	`work/core.rs` (`Core::run_step`: il lavoro che va in panic esce con `Exit::Panicked`), `editor/boot.rs` (`EditorState::new`, che sottomette evidenziatore e prescansione, e `create_global_env`, che sottomette controllo dei file e salvataggio automatico, una volta sola)
Verifica	Riprodotto sulla revisione `3d846f1` con un lavoro Lisp: dopo `(background-call 'boom (lambda () (set-face 'region "#aéb12")))` il registro dice «[BKG] panic: boom retired», il messaggio del panic va su stderr e un worker contatore continua a contare. Per i lavori dell'editor dalla lettura del codice.

Cosa succede. Da quando BKG-1 è risolta, un panic in un passo ritira solo il lavoro che l'ha causato e lo scrive nel registro con il tag `[BKG]`. Per un lavoro Lisp basta. I lavori dell'editor, evidenziatore, prescansione, controllo dei file e salvataggio automatico, sono però sottomessi una volta sola all'avvio, e un lavoro ritirato non torna: dopo un panic dell'evidenziatore la colorazione si ferma fino alla fine della sessione, e l'area messaggi non dice nulla. Un panic che avvenisse con un lock dell'editor in scrittura lo lascerebbe inoltre avvelenato, e il thread dei comandi andrebbe in panic al primo `.expect("write lock on …")`; oggi non si conosce un panic che avvenga sotto un lock.
Effetto. La colorazione, o il salvataggio automatico, smettono di funzionare senza che chi usa l'editor lo sappia.
Correzione proposta. Risottomettere i lavori dell'editor ritirati, con un limite perché un panic che si ripete non diventi un ciclo, e dire nell'area messaggi quale lavoro si è fermato; per i lock, decidere se uno avvelenato si può riprendere (`PoisonError::into_inner`) o deve chiudere l'editor in modo ordinato.

BKG-3 — `setenv` cambia l'ambiente del processo mentre il codice C di un altro thread può leggerlo

Gravità	BASSA
Dove	`primitives/environment.rs` (`setenv`)
Verifica	Dalla lettura del codice e della libreria standard di Rust.

Cosa succede. `std::env::set_var` e `remove_var` prendono un lock che prendono anche `std::env::var` e l'avvio di un processo con `std::process::Command`. Le letture dell'editor passano tutte di lì, anche sugli altri thread: un lavoro Lisp sull'esecutore che chiama `getenv` o `shell-command-to-string`, il salvataggio automatico che espande `~`, `stop shell` che avvia `/bin/sh` per il `KILL`. Il lock non copre il codice che legge l'ambiente da sé, con il `getenv` della libreria C, ed è per questo che `set_var` è `unsafe`. Nella TUI non c'è codice del genere su altri thread; nel front-end grafico le librerie di sistema che eframe carica (OpenGL, X11, Wayland) leggono variabili d'ambiente, e niente promette che non lo facciano su un loro thread. `setenv` stesso può girare sull'esecutore, chiamato da un lavoro Lisp. Fino al passo 17 della documentazione il commento SAFETY di `setenv` diceva che gli altri thread leggevano l'ambiente solo per avviare un processo dal thread dei comandi.
Effetto. Nel front-end grafico, un `setenv` mentre una libreria di sistema legge l'ambiente è comportamento indefinito: con glibc può far cadere l'editor. Non è stato osservato.
Correzione proposta. Tenere le variabili dei processi in una tabella dell'editor, letta da `getenv` e passata a ogni `Command` con `env_clear` ed `envs`, senza cambiare mai l'ambiente del processo: `setenv` non avrebbe più bisogno di `unsafe`.

8 Modi, sintassi e colorazione

SIN-4 — `make-mode` su un modo esistente lo sostituisce con uno vuoto

Gravità	BASSA
Dove	`primitives/modes.rs` (`make-mode`), `editor/modes.rs` (`EditorState::set_mode`), `managers/modes.rs` (`Modes::insert`)
Verifica	Dalla lettura del codice.

Cosa succede. Rivalutare il file di un modo (per esempio dopo una modifica) azzera la sua mappa dei tasti, i suoi hook, la grammatica e la tabella di sintassi, compresi quelli che altri moduli avevano aggiunto (come l'hook di `electric-pair`). La docstring di `make-mode` e i limiti della pagina `writing-modes` ora lo dicono.
Correzione proposta. Se il modo esiste già, restituirlo senza modificarlo, oppure svuotare solo grammatica e tabella.

SIN-5 — Il registro delle facce cresce per sempre e l'indice è a 16 bit

Gravità	BASSA
Dove	`ui/faces.rs` (`Face::intern`, `Face::all`)
Verifica	Dalla lettura del codice.

Cosa succede. Ogni nome di faccia mai usato resta registrato. `(names.len() - 1) as u16` si avvolge dopo 65 536 nomi distinti, e un nome nuovo prenderebbe l'indice di uno vecchio. I nomi arrivano anche da Lisp mentre l'editor gira (`set-face`, `face-style`, le facce di overlay e testo virtuale), quindi il commento che dice il registro limitato dalla configurazione è ottimista.
Correzione proposta. Un controllo con errore oltre `u16::MAX`, o un indice a 32 bit.

9 Ricerca, sostituzione e liste di risultati

RIC-5 — I risultati di occur, grep e compile puntano a righe che si spostano

Gravità	BASSA
Dove	`text/results.rs` (`Entry`, `Results::step`), `primitives/results.rs` (`visit`, `visit_index`, `refresh`)
Verifica	Riprodotto sulla revisione `3d846f1`: in `x`, `foo`, `bar`, dopo `(occur--scan "foo")` e una riga aggiunta in cima, `next-error` porta il punto all'inizio della riga di `x`.

Cosa succede. Ogni risultato conserva riga, colonna e offset come numeri, e `visit` si sposta per riga e colonna. Dopo una modifica che aggiunge o toglie righe sopra un risultato, `next-error` porta alla riga sbagliata; Emacs usa marcatori che seguono il testo. Il solo modo di aggiornare una lista, `results-refresh`, la ricostruisce da capo, e lo usa solo `compile` con l'uscita del compilatore.
Correzione proposta. Marcatori nei buffer aperti, aggiornati dalle due porte delle modifiche.

RIC-7 — Una lista di `M-s o` e una di `M-s g` aprono due strisce, una sopra l'altra

Gravità	BASSA
Dove	`editor/results.rs` (`display_results`, che cerca la striscia da riusare), `primitives/occur.rs` (`occur--scan` e `grep--scan`, che si presentano come produttori diversi)
Verifica	Riprodotto nell'editor sulla revisione `3d846f1`: una lista di `M-s o` e poi una di `M-s g` lasciano tre finestre, la lista di grep in una striscia nuova sopra quella di occur. Nessun test prova una lista di un tipo seguita da una dell'altro.

Cosa succede. `display_results` riusa la striscia in basso solo se la lista che vi è mostrata viene dallo stesso produttore, e occur e grep sono due produttori, `"occur"` e `"grep"`, anche se le loro liste sono entrambe in `occur-mode`. Prima di `3d846f1` una seconda lista, di qualunque dei due comandi, prendeva la striscia della prima.
Effetto. Chi cerca prima nel buffer e poi nella directory si ritrova con lo schermo diviso in tre, e deve chiudere la striscia in più con `q`. La pagina `searching` lo dice tra i limiti.
Correzione proposta. Se le due liste devono condividere la striscia, scegliere la striscia da riusare per tipo di lista (il modo del buffer) invece che per produttore, oppure dare a occur e a grep lo stesso produttore; in entrambi i casi un test con una lista di `M-s o` seguita da una di `M-s g`, e la riga tolta dai limiti della pagina.

RIC-8 — Un `query-replace` lasciato con un altro tasto resta aperto, con la corrispondenza evidenziata

Gravità	BASSA
Dove	`primitives/replace.rs` (`default_view`, che installa la domanda come mappa `OnUnbound::Release`), `editor/replace.rs` (`EditorState::end_replace`, che in quel caso nessuno chiama)
Verifica	Riprodotto con un test temporaneo durante il passo 13 della documentazione, sul codice di `9fcd84b` con i passi 12 e 13, che non cambiano il comportamento. Su `cat cat`, dopo `(query-replace "cat" "dog")` e il tasto `z`, la domanda sparisce e il testo è `zcat cat`; ma la sostituzione è ancora aperta, l'overlay `replace-match` copre ancora `cat` e `(replace-match)` risponde `(0 3 "zca" "dog")`. Dopo il tasto `y`, che ora inserisce una `y`, `(replace-this)` trasforma `zycat cat` in `dogat cat`.

Cosa succede. La vista predefinita installa la sua domanda come keymap transitoria `OnUnbound::Release`: un tasto che non è una risposta toglie la mappa e prosegue verso le altre keymap. La mappa se ne va dentro la risoluzione del tasto, che non ha un modo di chiudere ciò che c'è dietro, e nessuno chiama `end_replace`: la sessione resta in `Runtime`, l'overlay resta sul buffer, e le posizioni della sessione non seguono le modifiche successive. È la regola che `TransientKeymap` dichiara, una mappa con una sessione dietro è una `Refuse`, a non essere seguita.
Effetto. La corrispondenza resta evidenziata dopo che la domanda è sparita, finché un'altra sostituzione nello stesso buffer non toglie l'overlay; la pagina `searching` dice che un altro tasto ferma la sostituzione. I verbi della sessione restano attivi: chiamati da Lisp, per esempio da una vista scritta dall'utente, lavorano su posizioni vecchie, e `replace-this` sostituisce il testo sbagliato.

Correzione proposta. Chiudere la sessione quando la sua mappa se ne va: dopo ogni comando, fuori dalla risoluzione dei tasti, chiamare `end_replace` se una sostituzione è aperta e la sua mappa non è più installata. Oppure fare della domanda una mappa `Refuse` che lega anche un'uscita, rinunciando a lasciarla continuando a scrivere. In tutti e due i casi un test che, dopo il tasto in più, controlli la sessione e l'overlay e non solo la domanda.

RIC-9 — `results-put` e `results-select` prendono un BUFFER nil come il buffer chiamato `nil`

Gravità	BASSA
Dove	`primitives/results.rs` (`results-put e results-select`, che prendono ogni simbolo come il nome del buffer)
Verifica	Riprodotto con un test temporaneo durante il passo 18 della documentazione, sul codice di `71e1554`, che non cambia il comportamento. In `*scratch*`, `(results-put "p" "o" '(("buffer" "*scratch*" 1)) nil)` risponde 1, ma `(results-buffer)` e `(results-count "*scratch*")` rispondono nil, e `next-error` dice «No search results to walk»; senza il quarto argomento la lista si attacca a `*scratch*`. `(results-select nil)` risponde nil, dove `(results-select)` sceglie `*scratch*`.

Cosa succede. Le due primitive leggono BUFFER come un nome quando è un simbolo, e nil è il simbolo `nil`. `results-put` attacca la lista a un buffer `nil` che non c'è, cioè a nessuno, ne fa la lista che `next-error` percorre e risponde con il numero delle voci come se fosse andata; `results-select` cerca una lista nel buffer `nil`. Le docstring dicono il buffer corrente se manca, e le domande sulle liste (`results-count`, `results-entry`, `results-state`) prendono nil come assente. Nessun modulo fornito con l'editor passa nil.

Effetto. Un modulo che passa a `results-put` il suo argomento facoltativo perde la lista in silenzio, e la risposta dice il contrario.

Correzione proposta. Prendere nil come assente, come fanno le domande sulle liste, e un test per ciascuna primitiva.

10 Comandi di shell

SHL-4 — Il nome di una pagina di manuale viene passato alla shell senza virgolette

Gravità	BASSA
Dove	`core/lisp/features/manpage.lisp` (`manpage--system-page`, che passa a `shell-command-to-string` il comando `man` con il nome dentro; `manpage--rsedit-page`, che costruisce il percorso della pagina concatenando il nome)
Verifica	Riprodotto sulla revisione `3d846f1`: `(manpage--system-page "printf;touch FILE")` crea FILE.

Cosa succede. `(format "man -w %s 2>/dev/null" name)` passa il nome così com'è a `/bin/sh -c`. Un nome con spazi o metacaratteri viene interpretato dalla shell: `printf(3)` dà un errore di sintassi, che si presenta come «No manual entry», e `a;b` esegue due comandi. Il nome arriva senza filtri da `M-x manpage` e da `C-h m`; da `manpage-at-point` (e da `K` in vim) è limitato ai caratteri di un simbolo, che comprendono `*`, espanso dalla shell. Anche il percorso della pagina interna viene costruito concatenando il nome, quindi `../` esce dalla directory dei dati.

Correzione proposta. Racchiudere il nome tra apici singoli, con gli eventuali apici interni preceduti da escape, oppure rifiutare i nomi che non sono `[A-Za-z0-9._+-]+`.

SHL-5 — Un comando di shell che ignora `TERM` sopravvive all'editor

Gravità	BASSA
Dove	`primitives/shell.rs` (`stop_process`: il `KILL` dopo `GRACE`, 2 s, da un thread fuori dal WorkManager), `src/main.rs` (`SHUTDOWN_WAIT`, 500 ms, e il commento di `ShutDownOnExit`)
Verifica	Riprodotto con un piccolo programma temporaneo durante il passo 17 della documentazione, sul codice di `9fcd84b` con i passi 12–16, che non cambiano il comportamento: un editor avvia un comando, lo fa girare mezzo secondo, chiama `shutdown` con l'attesa di `src/main.rs` ed esce. Con `sleep 37` il processo non c'è più subito dopo; con `trap '' TERM; exec sleep 38` è ancora vivo tre secondi dopo l'uscita. Con l'editor aperto, `stop-worker` sullo stesso comando lo toglie entro tre secondi.

Cosa succede. Fermare un comando manda `TERM` e `CONT` al suo gruppo e avvia `stop shell`, un thread fuori dal WorkManager che dopo 2 s manda `KILL` a ciò che resta. All'uscita `shutdown` ferma i comandi così e aspetta al più 500 ms; poi il processo dell'editor finisce, e `stop shell` con lui, prima del `KILL`. Un comando che ignora `TERM`, o che lo gestisce senza uscire, resta vivo, in un gruppo che nessun terminale guarda. Il commento di `ShutDownOnExit` dice che un comando non sopravvive all'editor.

Effetto. Un processo dimenticato, che continua a usare CPU, file o porte dopo l'uscita.

Correzione proposta. All'uscita mandare `KILL` a ciò che resta dei gruppi dei comandi, dopo l'attesa di `shutdown`; oppure fare del `KILL` ritardato un lavoro del WorkManager, che `shutdown` aspetta.

11 Disegno e terminale

TUI-3 — Le tabulazioni non vengono espanse

Gravità	MEDIA
Dove	`ui/layout.rs` (`Row::plain`, `compose_row`, `Row::to_screen`, `Row::to_buffer`: una colonna del buffer è una colonna dello schermo), `src/tui/render.rs` (`render_to`, `draw_clipped_row`)
Verifica	Riprodotto nell'analisi di `08a853d`: la riga della fotografia conteneva il carattere `\t`. `ui/layout.rs` non è cambiato da allora.

Cosa succede. L'editor conta un tab come una colonna. Il terminale invece lo espande fino alla tabulazione successiva, quindi il resto della riga viene disegnato più a destra di dove l'editor crede. Il commento in cima a `text/rectangle.rs` dice che un tab è una colonna, come in tutto l'editor: è il modello interno, non ciò che accade sullo schermo. Esiste ora un'impostazione `tab-width`, nel modulo `indent`, ma serve solo al rientro: il disegno non la legge.
Effetto. In un Makefile, in un file Go o in qualsiasi sorgente indentato con tab, il cursore appare nel punto sbagliato, i colori cadono sulle parole sbagliate e il resto della riga precedente può restare visibile.
Correzione proposta. Espandere i tab in spazi nella composizione delle righe (`compose_row`), con una larghezza configurabile, e usare la stessa mappa per il cursore e per i clic del mouse.

TUI-4 — I caratteri larghi contano come una colonna

Gravità	MEDIA
Dove	`ui/layout.rs` (`Row::plain`, `compose_row`, `Row::to_screen`, `Row::to_buffer`), `src/tui/render.rs` (`render_to`, che riempie le righe contando i caratteri)
Verifica	Dalla lettura del codice: le larghezze sono tutte `chars().count()`, e nessuna dipendenza calcola la larghezza di visualizzazione (`unicode-width`).

Cosa succede. Un ideogramma o un'emoji occupa due celle del terminale, ma l'editor ne conta una.
Effetto. Come per i tab: cursore, evidenziazioni, ritaglio e riempimento delle righe sfasati nei testi in cinese, giapponese, coreano o con emoji.
Correzione proposta. Calcolare la larghezza con `unicode-width` nella stessa mappa di TUI-3.

TUI-5 — Un colore esadecimale con caratteri non ASCII manda l'editor in panic

Gravità	MEDIA
Dove	`ui/faces.rs` (`Color::parse`), chiamata da `primitives/theme.rs` (`colour_arg`, dietro `set-face` e `define-face`)
Verifica	Riprodotto sulla revisione `3d846f1`: `(set-face 'region "#aéb12")` va in panic con «end byte index 2 is not a char boundary».

Cosa succede. `Color::parse` controlla la lunghezza in byte e poi taglia la stringa con `&hex[0..2]`. Con un carattere di due byte il taglio cade a metà del carattere.
Effetto. Chiusura dell'editor, con il terminale in raw mode (TUI-2). Il caso si presenta anche con un file di tema sbagliato. In un lavoro di background il panic ritira solo quel lavoro.
Correzione proposta. Rifiutare i colori che non sono ASCII (`hex.is_ascii()`) prima di tagliare.

TUI-6 — Le finestre flottanti non mostrano colori di sintassi né overlay

Gravità	BASSA
Dove	`editor/frame.rs` (`EditorState::snapshot`, il ciclo sulle finestre flottanti), a confronto con `ui/compose.rs` (`LayoutNode::compute_tiled_views`)
Verifica	Dalla lettura del codice: per una finestra flottante si raccolgono solo la regione e il testo virtuale.

Cosa succede. Un buffer mostrato in una finestra flottante appare senza colori e senza le evidenziazioni degli overlay; lo stesso buffer in una finestra affiancata le ha.
Correzione proposta. Usare per le finestre flottanti la stessa raccolta delle evidenziazioni delle finestre affiancate.

TUI-7 — `%p` nella riga di stato dice `All` solo per un buffer di una riga

> **Gravità** BASSA
> **Dove** `ui/mode_line.rs` (`position_in_buffer`, ed `expand_mode_line`, che non riceve l'altezza della finestra)
> **Verifica** Dalla lettura del codice.

Cosa succede. Il codice verifica `lines <= 1`: `expand_mode_line` non riceve l'altezza della finestra, e non può sapere se il buffer intero sta in una schermata, che è quando Emacs dice `All`. Il commento di `position_in_buffer` diceva il contrario; dal passo 20 della documentazione descrive il comportamento di oggi. Il test `the_position_reads_as_emacs_words_it` lo fissa.
Correzione proposta. Passare l'altezza della finestra e confrontare con quella.

TUI-8 — La TUI non usa lo schermo alternativo

> **Gravità** BASSA
> **Dove** `src/tui/mod.rs` (`tui_main`), `src/tui/render.rs` (`render_to`)
> **Verifica** Dalla lettura del codice: nessun `EnterAlternateScreen`.

Cosa succede. L'editor disegna sullo schermo principale del terminale e all'uscita lo cancella con `Clear(All)`.
Effetto. Il contenuto che il terminale mostrava prima di avviare l'editor, per esempio l'output di un comando, va perso.
Correzione proposta. `EnterAlternateScreen` all'avvio e `LeaveAlternateScreen` all'uscita, nella stessa guardia di TUI-2.

12 La finestra grafica

GUI-1 — La finestra grafica si aggiorna ogni 250 ms anche quando nulla cambia

> **Gravità** BASSA
> **Dove** `src/gui/app.rs` (`EditorApp::update`, `BACKGROUND_POLL`)
> **Verifica** Dalla lettura del codice, durante il passo 20 della documentazione.

Cosa succede. La TUI registra `on_background_change`, e l'editor la sveglia quando un lavoro di sfondo cambia qualcosa; la GUI no. Per vedere ciò che i lavori fanno chiede a egui un aggiornamento entro `BACKGROUND_POLL`, 250 ms, anche quando `next_redraw_in` risponde `None`. A ogni aggiornamento chiama `tick`, compone la fotografia con `snapshot` e la ridisegna, anche con l'editor fermo.
Effetto. Una finestra aperta e ferma consuma CPU, e un portatile la batteria, quattro volte al secondo; ciò che un lavoro di sfondo cambia compare fino a 250 ms dopo.
Correzione proposta. Registrare con `on_background_change` una callback che chiami `request_repaint` sul contesto di egui, e chiedere un aggiornamento solo quando `next_redraw_in` o `drag_scroll_in` danno una durata.

13 Documentazione, commenti, codice morto, build e test

DOC-1 — Commenti e docstring che descrivono un comportamento che non c'è più

> **Gravità** BASSA
> **Dove** vedi l'elenco
> **Verifica** Dalla lettura del codice.

Cosa succede.

- `primitives/shell.rs` (`shell-command-start`): il messaggio «The editor's background work has shut down» vale anche quando il sistema non dà un thread nuovo, l'altro caso in cui `WorkManager::spawn` rifiuta. Il commento accanto, che diceva lo stesso, è corretto dal passo 17 della documentazione.
- `primitives/occur.rs` (`grep--scan`): il messaggio «The editor's background work has shut down» vale anche quando il sistema non dà un thread nuovo, l'altro caso in cui `WorkManager::spawn` rifiuta. Il commento accanto, che diceva lo stesso, è corretto dal passo 18 della documentazione.
- `primitives/workers.rs` (`define-worker` e `background-call`, attraverso `start`): la riga del registro «the editor's work has shut down» vale anche quando l'esecutore non è partito, l'altro caso in cui `WorkManager::submit` rifiuta. Il commento accanto è corretto come quello di `shell.rs`.

La voce su `src/tui.rs` (`render_to`, il commento sull'area messaggi, che diceva il minibuffer agganciato alle ultime righe del frame) è corretta dal passo 20 della documentazione, e il commento è oggi in `src/tui/render.rs`. Le voci dell'analisi su `primitives/workers.rs` e su `shell-command-to-string` non valgono più: il codice e la docstring sono stati riscritti (`6de16a1`, `4f05134`). Le tre sui commenti della libreria di base dell'interprete sono nel rapporto di risp, dentro le schede INT-9, INT-16 e INT-21.

Correzione proposta. Aggiornarli insieme alle correzioni dei problemi a cui rimandano.

DOC-2 — Trappole latenti: `Debug` e `PartialEq` di `EditorState`

> **Gravità** BASSA
> **Dove** `editor/mod.rs` (`impl Debug for EditorState`, e accanto `impl PartialEq for EditorState`)
> **Verifica** Dalla lettura del codice.

Cosa succede. `Debug` per `EditorState` è un `todo!()` e `PartialEq` un `unreachable!()`: li richiede il trait `LispContext` di risp. Un `dbg!`, un `{:?}` o un `assert_eq!` fallito su una struttura che contiene uno stato dell'editor va in panic invece di stampare; oggi nessun tipo di risp contiene un contesto, e la trappola resta latente.

Correzione proposta. Un `Debug` che stampi almeno i nomi dei buffer, e un `PartialEq` per identità.

DOC-5 — Il codice delle liste di risultati lascia un metodo morto, due `if` da unire e righe da formattare

> **Gravità** BASSA
> **Dove** `editor/buffers.rs` (`free_buffer_name`), `editor/results.rs` (`select_results_target`, `quit_results`), e i file del codice toccato da `3d846f1`: `editor/results.rs`, `editor/settings.rs`, `editor/boot.rs`, `primitives/results.rs` e `primitives/occur.rs`, in cui il passo 18 della documentazione lo ha diviso, `primitives/shell.rs`
> **Verifica** Riprodotto sulla revisione `3d846f1`: ogni `cargo build` avvisa che `free_buffer_name` non è mai usato; `cargo clippy` dà tre avvisi in più della revisione precedente, `5f79477`; `cargo fmt --check` trova 41 blocchi da riformattare in quei cinque file, dove prima ne trovava 3, tutti in `src/gui.rs`.

Cosa succede. Con `3d846f1` le liste di occur si aprono con `prepare_results_buffer`, che sceglie il nome con `free_name_for_work`: se ne va così l'ultima chiamata a `free_buffer_name`, e il metodo resta, con la sua docstring. In `select_results_target` e in `quit_results` un `if let` contiene un solo `if`, che clippy chiede di unire (`collapsible_if`). Il codice nuovo non è passato da `rustfmt`: righe spezzate dove ci starebbero intere, la registrazione di `results-display` e `results-quit` con l'indentazione persa, spazi in fondo a una riga di `primitives/shell.rs`.

Effetto. Nessuno per chi usa l'editor. Un avviso fisso in ogni compilazione nasconde quelli nuovi, e la prossima modifica a questi file, passata da `rustfmt`, porterà con sé decine di righe che non c'entrano.
Correzione proposta. Togliere `free_buffer_name`, unire i due `if`, passare `cargo fmt` sui cinque file, in un commit che non cambia altro.

14 Appendici

14.1 Le schede risolte

Le schede risolte dopo l'analisi, con il commit che le ha risolte. Il testo completo di ciascuna sta nella storia di git di questo file; che cosa è cambiato, e perché, nel messaggio del commit.

ID	Problema	Gravità	Risolto in
BUF-1	`undo` e `redo` modificano il testo senza passare dalle due porte delle modifiche	MEDIA	`000b0a8`
BUF-2	`cursor_pos()` costa O(posizione): ogni fotogramma rilegge il buffer fino al punto	MEDIA	`64e33a3`
CMD-1	I caratteri non ASCII non si possono digitare	MEDIA	`b9873db`
CMD-2	Mancano Canc, Inizio, Fine, PagSu, PagGiù e i tasti funzione	MEDIA	`3fb5e18`
CMD-3	Argomenti numerici enormi: allocazioni che terminano il processo e panic	MEDIA	`d86f0ea`
MIN-1	Due domande aperte insieme condividono `*Minibuffer*` e lasciano una finestra fantasma	ALTA	`b8829ad567399b388a0bc9dedf51`
MIN-2	`C-g` dentro il minibuffer non annulla la domanda	MEDIA	`041145b`
MIN-3	Il contenuto del minibuffer viene sostituito senza passare dalle porte delle modifiche, e solo fino al punto	BASSA	`5a89afd`
MIN-5	`minibuffer-choose-completion` scrive nel buffer corrente, anche quando non è il prompt	MEDIA	`5a89afd`
FIL-1	Un salvataggio fallito con `C-x C-s` non viene segnalato	ALTA	`7d922071baf5fa4d98fbf1c02654`
FIL-2	I file vengono scritti sul posto: un errore a metà li lascia troncati	MEDIA	`fdf930d`
FIL-3	Un file non UTF-8 non si apre e non viene detto nulla	MEDIA	`de075c1`
AVV-1	L'editor non parte se la directory corrente non è scrivibile	ALTA	`58d0b0d`
AVV-2	Gli errori di sintassi nei file Lisp caricati vengono ignorati in silenzio	MEDIA	`7433e5d`
AVV-3	Un errore di scrittura del registro manda l'editor in panic	MEDIA	`a20f050`
AVV-4	Il registro in memoria cresce senza limite	BASSA	`a20f050`
AVV-5	La configurazione predefinita viene scritta solo se manca	BASSA	`d86f0ea`
AVV-7	Un errore in `init.lisp`, o in un modulo che richiede, impedisce all'editor di partire	MEDIA	`7433e5d`
BKG-1	Un panic in un lavoro di background uccide per sempre il thread condiviso	ALTA	`6de16a1`
SIN-1	Una regione di sintassi che corrisponde alla stringa vuota blocca il thread di background	MEDIA	`173d8f5`
SIN-2	Colorare una riga molto lunga costa un tempo quadratico nella sua lunghezza	MEDIA	`173d8f5`
SIN-3	`write-file` che cambia il modo lascia i colori del modo precedente	BASSA	`cf0d8ac`
SIN-6	`forward-sexp` dall'interno di un simbolo salta alla fine dell'espressione successiva	MEDIA	`faa4694`
SIN-7	Una regola con GROUP colora tutta la corrispondenza quando il gruppo non vi prende parte	BASSA	`173d8f5`
SIN-8	`forward-sexp`, `kill-sexp` e `down-list` alla fine di una lista proseguono nelle liste che seguono	MEDIA	`faa4694`
SIN-9	`forward-sexp` si ferma alla fine di un commento a blocchi ma scavalca un commento di riga	BASSA	`faa4694`
RIC-1	Una sostituzione con un'espressione che può essere vuota mette tutte le sostituzioni nello stesso punto	MEDIA	`3f1df42`
RIC-2	La ricerca all'indietro con un'espressione regolare costa un tempo quadratico	MEDIA	`3f1df42`
RIC-3	Ogni sostituzione è un gruppo di annullamento a sé	BASSA	`3f1df42`
RIC-4	Limiti della ricerca confinata a una regione e delle scansioni su righe lunghe	BASSA	`3f1df42`
RIC-6	Alla fine di una ricerca incrementale il mark non è dove la ricerca è cominciata	BASSA	`1aa5cf7`
RIC-10	Il segno di un risultato in un file resta quando la visita passa a un altro file	BASSA	`287a2eb`

ID	Problema	Gravità	Risolto in
SHL-1	Un comando che scrive molto su stderr si blocca	MEDIA	`02ea0e7`
SHL-2	Un solo byte non UTF-8 nell'output interrompe il comando	MEDIA	`02ea0e7`
SHL-3	Non c'è modo di fermare un comando in corso	BASSA	`02ea0e7`
TUI-1	I caratteri di controllo del testo arrivano al terminale: un file può comandarlo	ALTA	`543ef5f737128145564112ee107c0`
TUI-2	Un panic o un errore di I/O lasciano il terminale in raw mode	ALTA	`67a6a1d7a4e09d2854c57137dcaa5`
DOC-3	Lo script di build presume dove si trova la directory `target`	BASSA	`f742701`
DOC-4	Un test di prestazioni instabile e clippy che fallisce sui test	BASSA	`10598fa`

14.2 Differenze da Emacs che non sono difetti

Alcuni comportamenti differiscono da Emacs per scelta, o comunque senza produrre risultati sbagliati. Sono elencati qui perché chi conosce Emacs non li scambi per errori. Quelle dell'interprete sono nel rapporto di risp.

- Le espressioni regolari usano la sintassi del crate `regex` di Rust, non quella di Emacs. `\(` è una parentesi letterale e i gruppi si scrivono `( )`. Non esistono lookaround né riferimenti all'indietro, ed è proprio ciò che garantisce un tempo lineare.
- Le liste di risultati, le ricerche e le sostituzioni non tornano mai all'inizio da sole: quando la lista finisce lo dicono, come scelta esplicita di progetto.

14.3 Parti esaminate e trovate solide

Nell'analisi queste parti sono state esercitate di proposito sui casi limite senza trovare problemi:

- lo scanner delle espressioni bilanciate (`modes/sexp.rs`): stringhe con escape alla fine del buffer, parentesi sbilanciate in entrambe le direzioni, letterali carattere, lifetime di Rust, stringhe raw, commenti a blocco annidati, commenti che contengono parentesi. Fuori da questi casi, più tardi, le verifiche hanno trovato SIN-6 e SIN-8
- la ricerca incrementale: parentesi letterali, espressioni non valide, cancellazione, ricerca fallita e ripresa dopo il fallimento;
- kill e yank, compreso `M-y`, e le macro di tastiera (con il limite alla ricorsione delle macro);
- `kill-buffer` di un buffer mostrato in due finestre, la chiusura dell'unica finestra, la chiusura di `*scratch*`;
- la sostituzione `a` → `aa`, che termina;
- i comandi di modifica ai bordi del buffer;
- `delete-file` sui link simbolici a directory, che cancella il link e non il contenuto della directory di destinazione;
- l'uscita con buffer non salvati: dopo un salvataggio fallito il buffer resta segnato come modificato e l'editor chiede conferma prima di uscire.